

The SIC-2 Architecture: Design, Instruction Set, and Educational Significance

Introduction and Historical Context

The *Simplified Instructional Computer* (SIC) is a hypothetical computer architecture introduced by Leland L. Beck in his textbook “*System Software: An Introduction to Systems Programming*.” Beck developed SIC to strip away the complexities of modern processors and provide a clear, accessible model for teaching system software concepts ¹. The original SIC (sometimes called *SIC Standard Model*) includes only the most essential hardware features common to real machines, making it easier for students to learn how assemblers, linkers, and operating systems interact with computer architecture ¹. In response to the need for a more capable yet still educational platform, Beck also defined an enhanced version of this machine. This enhanced architecture – referred to here as **SIC 2** (and known in Beck’s text as **SIC/XE**, for “eXtended Equipment” or “Extra Expensive”) – builds upon the standard SIC by adding significant features and resources while maintaining backward compatibility ². Object programs assembled for the original SIC can execute correctly on SIC 2 (SIC/XE), demonstrating full upward compatibility between the two versions ³ ⁴.

The creation of SIC and SIC 2 was motivated by educational needs. By the late 20th century, real CPUs had grown complex – with pipelines, caches, microcode, and elaborate instruction sets – making them less ideal for beginner-level system programming exercises. Beck’s SIC architecture abstracts away those complexities in favor of clarity ¹. The later SIC 2 variant introduces more advanced architectural concepts (such as additional addressing modes, registers, and I/O mechanisms) in a controlled manner, allowing students to transition into more realistic system-level programming without the full complexity of a commercial CPU. Historically, SIC (first appearing in Beck’s work around the 1980s) and SIC 2 have been widely adopted in academia as a sandbox for teaching assembly language, compiler design, and operating system fundamentals. These architectures are not implemented in silicon commercially, but they have been brought to life in simulators and even FPGA-based experimental “chips” in academic projects ⁵ ⁶. In the sections that follow, we provide an in-depth exploration of the SIC 2 architecture – covering its processor design, instruction set, addressing modes, execution model, memory and I/O organization – and we highlight the enhancements of SIC 2 over the original SIC. We also discuss the role of SIC 2 in computer architecture education, its use in simulation environments, and how it has been implemented in emulators for teaching and research.

Overall Architecture of SIC and SIC 2

Processor and Registers: Both SIC and SIC 2 are *single-accumulator*, binary, stored-program machines with a 24-bit word size. The processor of the standard SIC includes five core registers, each 24 bits long ⁷. These are:

- **A (Accumulator, reg# 0):** General-purpose accumulator used for arithmetic/logical operations ⁸. Many instructions implicitly use register A for data manipulation (e.g. arithmetic instructions take an operand from memory and combine it with A, leaving the result in A).
- **X (Index Register, reg# 1):** Used for indexed addressing; it holds an index value that can be added to addresses for data access (useful for array traversal, loop indexing, etc.) ⁸. X can also serve as a general counter or offset register in programs.
- **L (Linkage Register, reg# 2):** Used for subroutine linkage. The *Jump to Subroutine* instruction (JSUB) saves the return address (the next instruction's address) in L, and the *Return Subroutine* instruction (RSUB) jumps back to the address in L ⁹. In absence of a hardware stack, L provides a mechanism for subroutine returns; however, this means recursive or reentrant calls require software conventions to save/restore L if needed.
- **PC (Program Counter, reg# 8):** Holds the address of the next instruction to fetch ¹⁰. The PC is automatically updated as instructions are fetched and can be altered by branch/jump instructions. It functions like the instruction pointer in real CPUs, enabling sequential execution flow through memory.
- **SW (Status Word, reg# 9):** A status register containing condition codes and other status flags ¹⁰. In SIC, the SW's primary defined field is the *Condition Code (CC)*, which reflects the result of the most recent comparison or test (indicating "<", "=", or ">" condition) ¹¹ ¹². The CC is used by conditional jump instructions (like JLT, JEQ, JGT) to determine program flow ¹³. In SIC 2, the SW register may be expanded to include additional flags or mode bits (for example, some educational implementations include bits for supervisor/user mode, interrupt masks, and an *interrupt code* field to simulate basic OS support ¹⁴ ¹⁵). These additional status bits are not heavily used in Beck's original SIC examples aside from the CC, but they provide hooks for implementing operating system features (e.g., privilege levels or simple interrupt handling) in more advanced educational exercises.

SIC 2 extends the register set to include several new registers, reflecting features found in more advanced machines ¹⁶:

- **B (Base Register, reg# 3):** Introduced in SIC 2 as a dedicated base pointer for addressing ¹⁷. B enables *base-relative addressing* (described later) and allows the program to access memory beyond the reach of 12-bit displacements by using a register as a segment base. It essentially serves as a relocation register for one segment of memory, a concept similar to base registers in real-world architectures.
- **S (General Register, reg# 4) and T (General Register, reg# 5):** Two additional 24-bit general-purpose registers with no special hardware-defined usage ¹⁷. These can be used by the programmer (or compiler) for any needed purpose. S and T increase the CPU's general register pool, making SIC 2 a more versatile 24-bit two's-complement machine (contrasting with the minimalistic 1-accumulator design of the original SIC). For instance, S and T can hold intermediate values, loop counters, or serve as base addresses for data structures if the programmer chooses.
- **F (Floating-Point Accumulator, reg# 6):** A 48-bit register for floating-point arithmetic ¹⁸. This register (present only in SIC 2) enables the machine to handle a new numeric data type: 48-bit

floating-point numbers. The presence of F is accompanied by new floating-point instructions and represents a major enhancement over the integer-only SIC. The F register's larger size (48 bits) is designed to accommodate one standardized floating-point format in SIC 2.

It should be noted that the register numbers (codes) for PC, SW, and the new registers are not consecutive; PC and SW remain identified by numeric codes 8 and 9, respectively, preserving their special roles ⁹. Meanwhile, B, S, T, and F take the numeric codes 3, 4, 5, and 6 ¹⁹. This scheme ensures that none of the new registers conflict with the original ones. In assembly language, these registers are referred to by mnemonic (A, X, L, B, S, T, F, etc.), but their numeric identifiers are used in machine code and in certain instruction formats.

Memory Organization and Data Representation: Memory in both SIC and SIC 2 is a flat, byte-addressable array of bytes, with no separate I/O or specialty memory areas (unless defined by software conventions). On the standard SIC, the address space is **15 bits**, allowing for $2^{15} = 32,768$ **bytes** of memory ²⁰. Each memory address refers to a single 8-bit byte. Groupings of three consecutive bytes form a 24-bit *word*, which is the natural word size of the machine ²⁰. Words are aligned on any byte boundary (i.e., word addresses are defined by the lowest-numbered byte of the three-byte group) ²⁰. Because addresses are byte-oriented, incrementing an address by 1 moves to the next byte in memory, and reading a word from address N will fetch bytes at addresses N , $N+1$, and $N+2$ as a combined 24-bit value. By convention, these three bytes are typically interpreted in *big-endian* style (the byte at the lowest address is the high-order portion of the word) – this is implied by the definition that the “lowest numbered byte” is used as the word's address ²⁰, similar to many mainframe and RISC architectures.

In SIC, memory is limited to 32K bytes and addresses wrap around within that range. SIC 2 dramatically expands memory capacity to **20-bit addresses**, giving a maximum of $2^{20} = 1,048,576$ **bytes (1 MiB)** of memory ²¹. This expansion mirrors real trends in computing (where later architectures broke the 16-bit address barrier into 20-bit or 24-bit ranges). The larger memory necessitated new instruction formats and addressing modes, which we detail below. Aside from size, memory in SIC 2 remains byte-addressable and word-oriented in the same manner as SIC; the fundamental memory structure is unchanged ²². Both versions treat memory as a linear address space with no hardware-enforced partition between code and data. This means, for educational purposes, one can load instructions and data anywhere in memory (subject to program design), and the architecture doesn't impose protection or segmentation – those concepts can be introduced at a higher level if needed.

The **data formats** supported by SIC are deliberately minimal. The machine operates primarily on integer and character data:

- **Integer numbers** are 24-bit binary values, stored in a word (3 bytes). SIC uses standard two's complement representation for signed integers, allowing both positive and negative values in the range -2^{23} to $2^{23}-1$ ²³ ²⁴. There is no separate support for larger integers or for packed binary-coded decimals – only basic 24-bit binary arithmetic is directly provided by hardware.
- **Character data** uses the ASCII encoding, with one character per byte (8 bits) ²⁴. This choice reflects common practice and ensures that SIC's memory can represent text as well as numbers. Since a word is 3 bytes, up to three ASCII characters can fit in one word, though typical usage would treat character strings as byte arrays.

- **Floating-point** numbers are *not* supported in the standard SIC; any needed real-number calculations would have to be done in software (e.g., fixed-point or via simulation). This was a simplification to avoid dealing with complex floating hardware in the basic model ²⁵ ²⁴.

SIC 2 introduces a **48-bit floating-point format** to broaden the range of applications and to simulate a common feature of real machines ²⁶. The floating-point representation in SIC 2 is a custom 48-bit format consisting of a 1-bit sign, an 11-bit exponent, and a 36-bit fraction (mantissa) ²⁷. The exponent is stored with a bias of 1024 (i.e., exponents are in the range 0–2047, corresponding to actual exponents of –1024 through +1023) ²⁸. The fraction is a normalized fractional part (with an implied leading 1 for non-zero numbers, similar to IEEE normalization, though the base is 2). In effect, a floating-point number's value is:

$$(-1)^{\text{sign}} \times 1.\text{fraction}_{(\text{base}2)} \times 2^{(\text{exponent}-1024)}.$$

For example, an F-register value with sign 0, exponent 1024, fraction 0 represents +1.0; an exponent of 1024+10 with a given mantissa represents that number $\times 2^{10}$, etc. This format allows SIC 2 to handle a reasonable range of real numbers and perform floating arithmetic operations in hardware. The inclusion of floating-point in SIC 2 is a pedagogical step to expose students to FP operations and representation issues (e.g., precision and bias) without having to adopt the full IEEE 754 standard. In summary, SIC 2 supports integers and characters identically to SIC ²⁹, but adds a new numeric type (48-bit float) with corresponding instructions and the dedicated F register for computation.

Word Alignment and Endianness: The architecture does not impose strict alignment rules for words – any address can be the start of a word, though naturally word-aligned addresses (divisible by 3) are convenient. Given the educational nature, alignment issues are typically glossed over. Endianness, as mentioned, is effectively big-endian for word access (address points to the high-order byte). To illustrate: if the word value 0x123456 is stored at memory address 1000, then memory[1000]=0x12, memory[1001]=0x34, memory[1002]=0x56 (in hex). If the CPU loads a word from address 1000 into register A, A will contain the value 0x123456. This scheme is consistent across SIC and SIC 2.

Instruction Set and Addressing Modes

Instruction Formats: All SIC instructions are encoded in a fixed 24-bit format (3 bytes per instruction) ³⁰. The basic format can be visualized as follows:

- **Opcode:** 8 bits
- **X (index flag):** 1 bit
- **Address:** 15 bits

In the standard SIC, the 8-bit opcode identifies the operation (e.g., LOAD, ADD, JUMP, etc.), and the 15-bit address field provides the operand's memory address. The single **X flag** bit (the 24th bit of the instruction) indicates indexed addressing; if x=1, the machine will add the contents of register X to the given address to compute the effective address (this supports simple index-based memory access) ³¹. If x=0, the address field is taken as an absolute address (direct addressing). Thus, the standard SIC has two addressing modes for memory operands: *direct* and *indexed*, distinguished by the X bit ³². The **target address (TA)** calculation in SIC is straightforward:

- If x = 0: TA = *address* (the 15-bit address field zero-extended to 24 bits).
- If x = 1: TA = *address* + (X) (the content of index register X added to the address field) ³².

All memory-reference instructions in SIC follow this format. (We note that not all SIC instructions have a memory operand; a few, like *RSUB*, ignore the address field, but they still occupy the same 24-bit format in memory. *RSUB* simply sets $PC \leftarrow L$ to return from a subroutine, and its address field is typically filled with zeros by the assembler.)

SIC 2 greatly expands the variety of instruction formats to accommodate new types of instructions (register-to-register operations, larger addresses, etc.) ³³ ³⁴ . SIC 2 defines **four** instruction formats: Format 1, 2, 3, and 4 ³⁴ . A summary of these formats is given in **Table 1** below, which also shows how the 24-bit SIC format relates to the new format3/4 in SIC 2.

Table 1: SIC 2 Instruction Formats. Format 3 corresponds to the original SIC's 24-bit format extended with flag bits, and Format 4 is an extended 32-bit format for full 20-bit addressing ³⁴ ³⁵ .

- **Format 1:** A 1-byte instruction containing only an 8-bit opcode ³⁶ . Format1 instructions have no operand fields; the opcode alone completely specifies the operation. This format is used for simple, *implied-address* operations – typically machine control instructions or other operations that don't need a separate operand. For example, in SIC 2, the *FLOAT* (convert integer in A to floating in F) or *FIX* (convert F to integer in A) instructions, if they existed, could be Format1. More concretely, the I/O channel control instructions (HIO, SIO, TIO described later) are defined as Format1 in SIC 2 ³⁷ ³⁸ , since they use an implied operand (like the device or channel number in register A) and thus can be encoded solely by an opcode byte.
- **Format 2:** A 2-byte instruction, consisting of an 8-bit opcode and two 4-bit operand fields (often interpreted as register identifiers) ³⁶ . Format2 is used for instructions that involve register-to-register operations or other operations with small operand specifiers. In SIC 2, this format is employed by instructions such as **ADDR** (add registers, e.g., `ADDR R1, R2` adds the contents of R2 into R1), **SUBR**, **MULR**, **DIVR** (register-register arithmetic), as well as **COMPR** (compare registers) and data movement like **RMO** ("Register Move", copy one register's contents to another). Each of these instructions has two register operands encoded in the single extra byte (the first 4-bit field is R1, second 4-bit is R2) ³⁹ ⁴⁰ . Another group of Format2 instructions are those that operate on a register and a constant nibble; for instance, **TIXR** (an instruction to increment index and compare to a register) might be defined to use one register operand and one unused 4-bit field. In summary, Format2 handles all operations that deal primarily with registers rather than memory.
- **Format 3:** A 3-byte instruction format that extends the original SIC format with additional *flag bits*. In format3, the instruction layout is: 6-bit opcode, followed by six 1-bit flags, then a 12-bit *displacement* field ³⁴ ³⁵ . The total is 24 bits (3 bytes). The six flags (bits n, i, x, b, p, e) introduce powerful addressing mode features (detailed below). Format3 instructions are used for most memory-reference operations in SIC 2 when the target address can be expressed in 12 bits relative to some base or the PC. The 6-bit opcode space supports up to 64 opcodes, which is sufficient because many of the original SIC opcodes can be represented within that (the original had on the order of 25–30 distinct operations). Notably, by using 6 bits for the opcode, SIC 2 repurposes what used to be bits of the opcode in SIC for the new addressing mode flags. This clever encoding allows backward compatibility: legacy SIC instructions (which all had opcodes ending in binary `00`) can be treated as format3 with the flags implicitly indicating *simple addressing*. (In fact, original SIC code can be viewed as format3 with the n,i flags both 0, which SIC 2 interprets as the same direct addressing semantics as old SIC ⁴¹ ⁴² . We will explain n,i flags shortly.) The 12-bit displacement field in format3 is interpreted as a signed integer when used with PC-relative addressing (allowing jumps or references ± 2048 bytes from the current PC) or as an unsigned offset when used with base-relative addressing

⁴³ ⁴⁴ . If neither base nor PC-relative is enabled (b=0, p=0), the 12-bit field is taken as an absolute address (allowable only for addresses < 4096) ⁴⁵ . Format3 is the workhorse format for most typical instructions on SIC 2, as it offers a compact encoding for local jumps and data access while supporting multiple addressing modes.

- **Format 4:** A 4-byte instruction (32 bits) that is essentially an augmented version of format3. It has the same layout of fields except that the address field is 20 bits long instead of 12 (and the e flag bit is set to 1 to indicate format4) ³⁵ . Specifically: 6-bit opcode, n,i,x,b,p,e flags, and a 20-bit address. Format4 provides *extended addressing*: the full 20-bit address allows direct addressing of the entire 1 MB memory space of SIC 2 without relying on base or PC relative modes ³⁴ ⁴⁶ . In assembly language, format4 instructions are often denoted by a plus sign prefix (e.g., `+LDT #40980` might produce a format4 instruction to Load T with an immediate large constant). Format4 is used when a target address is out of reach for format3's 12-bit displacement or when the programmer explicitly wants an absolute address. Because it occupies an extra byte, format4 instructions should be used judiciously – the assembler or compiler might choose format3 by default and only use format4 when necessary to optimize code density, mimicking the idea of *long* and *short* instruction forms. All memory-reference opcodes that can be format3 have a corresponding format4 form (distinguished only by the e bit in machine code).

With four formats, SIC 2's instruction set achieves flexibility similar to real-world architectures like IBM System/360 (which also had 3- and 4-byte instructions for different addressing needs). Table 1 above illustrates the bit allocations in each format ³⁴ . Formats 1 and 2 are used for instructions that do not reference memory, whereas formats 3 and 4 are used for instructions that do reference memory (with the extended mode bit e distinguishing the 3-byte vs 4-byte variant) ³⁴ .

Addressing Modes in SIC 2: The addition of new flag bits in format3/4 gives SIC 2 a rich set of addressing modes beyond the basic direct and indexed modes of SIC. The six flag bits are: **n** (indirect), **i** (immediate), **x** (indexed), **b** (base-relative), **p** (PC-relative), and **e** (extended format indicator) ⁴⁷ ⁴⁸ . By setting combinations of these bits, the programmer can specify how the effective address of an operand is calculated. The roles of these flags are summarized below:

- **n (Indirect) and i (Immediate) flags:** These two bits determine the fundamental addressing *method* for the operand, akin to the addressing mode bits on some CISC machines. In SIC 2, there are three primary cases: *immediate*, *indirect*, and *simple/direct*. If **i=1 and n=0**, the instruction uses **Immediate addressing** – the operand value is taken *immediately* from the address field itself, rather than from memory ⁴⁹ . In other words, the 12- or 20-bit address field is not an address at all, but an actual constant value (e.g., `LDA #5` loads the literal value 5 into A, using immediate mode). The assembler indicates immediate mode by a `#` prefix on the operand. If **n=1 and i=0**, the instruction uses **Indirect addressing** – the address field gives the *location* of the real operand address ⁴⁹ . The machine will first fetch the word at the given address (plus indexing/base adjustments, if applicable) to obtain the actual target address, then fetch the operand from that target address. Indirect mode is indicated in assembly with an `@` prefix (e.g., `LDA @BUFFER` means load A from the memory location stored in the address labeled BUFFER). Finally, if **n=1 and i=1** (both bits set), this is interpreted as **Simple direct addressing** in SIC 2 ⁴⁹ ⁴¹ . In this case, the operand is taken directly from the effective address calculated (with base/PC/index applied as needed). This “n=i=1” combination yields the same behavior as the original SIC's direct addressing. The assembler by default produces n=1,i=1 for an operand written without `#` or `@` (unless it's an address that fits the original SIC's legacy pattern; but generally in SIC/XE assembly, unmarked operands are treated as

direct). Beck's design also cleverly uses the unused combination **n=0, i=0** for a special meaning: in original SIC object code, all opcodes had their two low bits 00 (which correspond to n=0,i=0 when interpreted in SIC 2 format3). This combination is treated as **"SIC compatibility mode"** or sometimes called *implied simple* addressing ⁴¹. Practically, n=0,i=0 is rarely generated by SIC 2 assemblers for normal instructions (since direct mode would be n=i=1), but the SIC 2 hardware considers n=0,i=0 as equivalent to n=1,i=1 (direct) for operand fetching ⁴¹ ⁴². This ensures that object code assembled under the old SIC rules (where those bits were essentially not used and remained 0) will still execute correctly, maintaining upward compatibility ³ ⁴. In addition, a few SIC 2 instructions use n=0,i=0 intentionally to denote *no operand at all* (for example, the machine control instructions like SVC or the channel I/O instructions might be represented with n=0,i=0 and no address, which effectively means the operand is "implied" by the instruction).

- **x (Indexed) flag:** This functions similarly to the original SIC's X-bit. If **x=1**, the content of register X is added to the computed address (after considering immediate/indirect and before the final memory reference) ³² ⁴¹. Indexed addressing can be combined with any of the above modes except where it doesn't logically apply (for example, indexing an immediate operand doesn't make sense, and indeed in SIC 2 the assembler will not allow `#` or `@` on the same operand as `,X`). The architecture simply defines that if x=1 and the addressing mode is *direct or base/PC relative*, X is added to the target address; if the addressing mode is immediate (n=0,i=1), the X flag is ignored or considered invalid (immediate values are not addresses). In summary, **Indexed** addressing in SIC 2 is an option that can augment direct, indirect, or base/PC-relative addressing (just as original SIC allowed indexing on its direct mode) ⁵⁰ ⁵¹. This is useful for accessing array elements or iterating through structures.
- **b (Base) and p (Program Counter) flags:** These two flags control **relative addressing** modes for the 12-bit displacement field in format3 instructions. They are mutually exclusive in most normal situations (it wouldn't make sense to have both b and p = 1 at the same time). If **p=1 (and b=0)**, the 12-bit displacement is interpreted as a signed offset relative to the current PC value (specifically, the PC after incrementing past the current instruction) ⁴³ ⁴⁴. This is **PC-relative addressing**, which allows efficient encoding of jumps or data references that are near the current instruction. The range is -2048 to +2047, since 12 bits with sign can represent offsets in that range. If **b=1 (and p=0)**, the 12-bit displacement is taken as an unsigned offset added to the content of the Base register B ⁵² ⁵³. This is **Base-relative addressing**. Base-relative mode requires that the programmer (or compiler/linker) load the B register with a suitable base address (often the start of a data area or the beginning of the program) before using B-relative instructions. The offset can range from 0 to 4095, covering any address in a 4KB window defined by B. Base-relative addressing is extremely useful for accessing static data or larger program ranges in SIC 2's expanded memory – by changing the base register, code can conveniently access different memory regions without needing 20-bit absolute addresses in each instruction. Only one of p or b is typically set for a given instruction: if **b=0 and p=0**, then no base register or PC relative addressing is used, and the **address field is interpreted as an absolute address** (direct addressing as in SIC, but now limited to 12-bit magnitude in format3) ⁴⁵. This case allows accessing the lowest 0–4095 memory range directly. If **b=1 and p=1** were to occur, it's generally undefined in normal usage (assemblers wouldn't produce that combination); we can assume the architecture would not use both simultaneously – one might imagine a scenario of p=1, b=1 being reserved or illegal. In practice, the assembler sets b or p depending on how it can resolve a given target address during code generation.

- **e (Extended) flag:** This flag is set to **1** for **Format4 (4-byte) instructions** and **0** for Format3 (3-byte) ⁵⁴. When e=1, the address field is 20 bits long and no PC-relative or base-relative displacement is used (the b and p bits are treated as part of the 20-bit address in format4). In other words, in format4, the instruction always uses **direct addressing with a full 20-bit address**, whether that address is target of a jump or a data reference. The n,i flags still apply (immediate vs indirect vs direct), and x can still be used to add indexing, but b and p are not meaningful in format4 (in fact, the hardware might simply ignore b,p when e=1, or assemblers enforce b=0, p=0 in that case). Format4 extended mode is typically employed for long jumps or references that cannot be encoded in 12 bits. For instance, `+JSUB PROCLONG` might generate a format4 instruction if the subroutine PROCLONG is far away in memory; this would allow a direct 20-bit jump. It comes at the cost of one extra byte in the instruction stream. All format4 instructions are explicitly indicated in assembly (commonly with a `+` as mentioned), ensuring the assembler sets e=1 appropriately. Format4 effectively gives an **Absolute addressing mode** spanning the full 1 MB address space ⁵⁵, whereas format3 relies on either base or PC relative to reach the higher addresses if possible.

Combining these flags yields the full set of addressing capabilities of SIC 2. To summarize the **addressing modes** available on SIC 2 (and how they compare to SIC):

- **Direct (Absolute) Addressing:** The simplest mode; the operand's address is explicitly stated (either in the 15-bit field for SIC or in the 12/20-bit field for SIC 2). In SIC 2 format3, this corresponds to n=1,i=1 with b=0,p=0 (and e=0) so that the 12-bit address is taken as absolute ⁴⁵. In format4, direct addressing is the only option (e=1 gives a 20-bit absolute address) ³⁵. Example: `LDA ITEM` (no special symbols) on SIC 2 might use PC-relative if ITEM is close, but if forced to absolute: `+LDA ITEM` would ensure a format4 absolute address. In SIC (original), all addressing was effectively direct unless X was used for indexing.
- **Indexed Addressing:** Not a separate mode by itself, but an option on top of direct or relative modes. In SIC, indexed was the only modification available (x flag) ³². In SIC 2, indexing (X) can be applied to direct, base, or PC-relative modes (but not to immediate or indirect). The effective address becomes *computed_address* + X. For example, `LDA ARRAY,X` will take the address of ARRAY, add X's content, and load A from that location. This is akin to using X as an array index register.
- **Immediate Addressing:** Operand is a literal constant embedded in the instruction (n=0,i=1). SIC did not support this mode (all SIC instructions that needed constants had to load from memory, or use a trick with *LDB* if one existed – but since SIC has no immediate, typically one would have to place constants in memory). SIC 2's immediate mode is a major convenience: e.g., `LDA #100` loads the value 100 into A without any memory reference besides the instruction itself ⁴⁹. This is useful for initializing registers or using small constants in arithmetic. The operand field is interpreted as an unsigned value (for the sake of assembly syntax, usually written as decimal or hex with a `#`). If the constant doesn't fit in 12 bits, the assembler might automatically choose format4 to supply a 20-bit immediate, or else load it from a literal pool.
- **Indirect Addressing:** Operand is accessed through a pointer. The address field gives the location where the actual target address is stored (n=1,i=0). SIC lacked this mode; pointers had to be handled by manual two-step loads (load address into a register, then use register-indirect via an indexed trick). In SIC 2, `LDA @PTR` will go to memory at address "PTR", fetch the 3-byte value there as the real address, then load A from that real address ⁴⁹. Indirect addressing allows dynamic data structures like linked lists or jump tables in a more straightforward way, and is another feature typical of real hardware (similar to "indirect" mode on PDP-11 or x86's pointer follows).

- **PC-Relative Addressing:** A mode where the address field is interpreted as a displacement added to the program counter ($n=1, i=1, p=1, b=0, e=0$) ⁴³. This is new in SIC 2 and improves code relocatability and density. For example, if a data variable is within 2KB of the instruction, the assembler can use PC-relative mode to refer to it, avoiding the need for a full 20-bit address or use of base register. The hardware will take the 12-bit displacement, sign-extend it to 24 bits, and add to the content of PC (which at that moment points to the next instruction). If the displacement is positive and small, this effectively encodes a forward reference; if negative (two's complement), it encodes a backward reference. *Jump* instructions heavily use PC-relative mode: e.g., `JEQ LOOP` can be assembled to a PC-relative jump if LOOP is within ± 2048 bytes, setting $p=1$ to encode the branch offset. This mode contributes to position-independent code principles and is analogous to the relative branches on many architectures (e.g., x86 short jumps or ARM PC-relative addressing).
- **Base-Relative Addressing:** Uses the base register B as a reference ($n=1, i=1, b=1, p=0, e=0$) ⁵². This mode is particularly useful for managing a larger program or data space. A common strategy is to load B with the start address of a data segment or the beginning of the program. Then instructions can reference any location in that 4K region via a 12-bit offset. For example, if B holds the address of an array `ARR`, an instruction `LDA ARR` might be assembled as base-relative ($b=1$) with $\text{disp} = 0$, meaning load A from $B+0$ (the start of ARR). Another data element 100 bytes into ARR could be referenced with $\text{disp}=100$. Base relative addressing thus provides a way to handle program relocation: if the program is loaded at a different address, the loader can simply set B accordingly, and all base-relative references automatically adjust. In educational OS projects, B might also serve as a general purpose base register for multi-segment memory management.
- **Extended (Format4) Addressing:** When $e=1$, the address is a full 20-bit absolute address, enabling direct addressing anywhere in the 1MB space. While not a separate addressing “mode” per se (it’s basically absolute addressing with a longer field), it’s often discussed as an *Extended mode*. In assembly, instructions that require the extended format are marked (e.g., writing `+JSUB X` forces a 4-byte JSUB). Extended format is needed if neither PC-relative nor base-relative can reach a target address – for instance, a far subroutine call or a far memory operand beyond the current 4K window. The trade-off is the instruction is one byte longer and slightly less efficient.

Additionally, SIC 2 retains the concept of **implied addressing** for certain instructions. Implied mode means the instruction implicitly operates on a fixed resource without any operand field. For example, *RSUB* in SIC (opcode 0x4C) has no operand; it always returns to L. In SIC 2, instructions like **HIO**, **SIO**, **TIO** (halt I/O, start I/O, test I/O channel) do not take an operand in the instruction – they implicitly refer to the device or channel specified in the A register. These would be encoded as Format1 instructions (just the opcode) and are documented as having “implied” addressing of A (or of an internal channel register). The SIC 2 assembler might forbid writing an operand for these. Another implied instruction example is a hypothetical *clear register* instruction, which might just specify the register in the opcode or format2 but no memory address.

Operation Sets and Example Instructions: The SIC and SIC 2 instruction sets cover several categories: data movement, arithmetic/logic, control flow (branching), subroutine linkage, and input/output, among others ⁵⁶ ⁵⁷. We will discuss these by category, highlighting enhancements in SIC 2:

- **Load and Store Instructions:** These move data between memory and registers. In SIC, we have instructions to load each of the standard registers from memory and to store them back. For example: **LDA** (opcode 0x00) loads a word from a given address into register A; **LDX** (0x04) loads the X register; **LDL** (0x08) loads the L register; **LDCH** (0x50) loads a single byte into the rightmost 8 bits of A (useful for reading character data) ⁵⁸. Corresponding store instructions include **STA** (0x0C,

store A's content to memory), **STX** (0x10), **STL** (0x14), and **STCH** (0x54, store the low byte of A to memory) ⁵⁸. There is also **STSW** (0xE8) to store the Status Word register to memory, often used before a supervisor call or interrupt to save program status ⁵⁹. SIC 2 introduces new load/store operations for the additional registers: **LDB** and **STB** for the base register B, **LDS/STS** for S, **LDT/STT** for T, and **LDF/STF** for the 48-bit floating-point register F ⁶⁰ ⁶¹. It also adds **LPS** (Load Processor Status, opcode D0) which presumably loads the SW register from memory (for restoring status during context switches, etc.) ⁶². All of these follow the standard addressing rules and can use immediate, indirect, etc., where applicable (though typically one loads from memory, not immediate, except LPS might allow immediate to set status flags). The existence of LDB/STS and friends highlights upward compatibility: original SIC code didn't have those, but they integrate seamlessly into SIC 2 for more flexibility.

- *Arithmetic and Logical Instructions:* SIC supports basic integer arithmetic using register A. **ADD** (0x18) adds a word from memory to A ($A \leftarrow A + [\text{memory}]$), **SUB** (0x1C) subtracts, **MUL** (0x20) multiplies, and **DIV** (0x24) divides (assuming DIV's opcode was 0x24; one source listed it as 0x34 ⁵⁷, but typically they are in sequence) ⁵⁷. These operations take the operand from memory (direct or indexed addressing) and leave the result in A ⁵⁷. Logical operations include **AND** (0x40) and **OR** (0x44), which perform bitwise conjunction or disjunction between A and a memory operand, storing the result in A ⁶³. SIC does not have a XOR or shift instructions in the base set – it's minimalist. **COMP** (0x28) compares the value in A with a word in memory and sets the condition code CC based on whether $A <$, $=$, or $>$ the memory operand ⁶³. Importantly, the arithmetic and logical operations do *not* automatically set condition codes in SIC (unlike some architectures where arithmetic sets flags). Only COMP and a few other specific instructions update the CC in SW ⁶³. The result of ADD, etc., is simply left in A, and overflow is not tracked (the SW has no documented overflow flag). If an overflow occurred in an addition, it would simply wrap around in two's complement arithmetic without an exception – again, a simplification for educational purposes. SIC 2 augments this category in several ways: Firstly, it adds **floating-point arithmetic** operations – **ADDF**, **SUBF**, **MULF**, **DIVF** – which perform their respective operations using the 48-bit F register and a 48-bit memory operand ⁴⁰. These allow arithmetic on floating point values analogous to the integer operations on A. Secondly, SIC 2 includes **register-to-register arithmetic**: **ADDR** (Add Register) and **SUBR**, **MULR**, **DIVR**, which add/subtract/multiply/divide the values of two registers (usually the result goes in the first operand register) ⁴⁰. For example, `ADDR B, S` would do $B \leftarrow B + S$ (assuming that definition) and would be a Format2 instruction. These make inner-loop arithmetic more efficient by avoiding memory access and mirror the kind of register operations found in modern RISC machines. Thirdly, SIC 2 likely adds **CLEAR** and **TIXR** instructions (common in SIC XE descriptions though not explicitly above): **CLEAR R** would set register R to 0 (often encoded as a special case of a register instruction), and **TIX** and **TIXR** we discuss under control flow. Logical instructions beyond AND/OR are not explicitly extended, but with more registers, one can do bit manipulations via register operations as well.

- *Comparison and Condition Code:* Besides the aforementioned **COMP** instruction (which compares A to a memory word) ⁶³, SIC has **TIX** (0x2C) – “Test and Increment Index.” The **TIX** instruction increments the content of register X by 1, then compares X to a given memory operand, setting the condition code accordingly ⁶⁴. This single instruction effectively performs a loop index update and loop termination check in one step. After TIX, CC will be `<` if $(X) < (\text{operand})$, `=` if equal, or `>` if $(X) > (\text{operand})$ ⁶⁴. A typical loop might load X with 0, store the loop limit in some memory location N, then use a label LOOP with TIX N and JLT (Jump if less than) back to LOOP – this would iterate X from

0 up to N-1. SIC 2 introduces **COMPR** (Compare Register) as a Format2 instruction to directly compare two registers and set CC (like `COMPR S, T` to compare S and T) – this avoids needing to store a register to memory just to compare. It also likely introduces **TIXR**, a variant of TIX that uses a register as the limit instead of a memory operand. TIXR would increment X and compare to some register, setting CC. These additions make looping and conditional logic more efficient. The CC in SW is also affected by I/O tests (see I/O section), but importantly, all conditional jump instructions in both SIC and SIC 2 (JLT, JEQ, JGT) rely on CC ¹¹ ¹³, so comparison outcomes control program flow.

- *Control Flow and Subroutine Instructions:* SIC provides a basic set of jump instructions and a subroutine linkage mechanism. The conditional jumps are **JLT** (Jump if CC is '<'), **JEQ** (if CC is '='), **JGT** (if CC is '>'). Each takes a target address as operand (like `JEQ SOMEWHERE` to branch to SOMEWHERE if last comparison was equal) ⁶⁵. There is also an unconditional jump **J** (0x3C) which sets PC to the target address unconditionally ¹², and the subroutine call **JSUB** (0x48) which does two things: stores the current PC in register L (thus saving the return address) and then jumps to the target address (similar to a JSR or CALL in other assembly) ¹². The counterpart **RSUB** (0x4C) returns from subroutine by jumping to the address contained in L ⁶⁶. As mentioned, RSUB has no operand field; it's effectively a "return" that always uses L. SIC does not have a dedicated *NOP* instruction, but a jump to the next address or a *TD* on a non-existent device could be used as a no-op if needed. In SIC 2, these same instructions exist but benefit from the new addressing modes. For example, jumps can use PC-relative displacement for smaller code. Additionally, SIC 2 adds **J** (unconditional) with extended format for far jumps if needed, and likely introduces a **RSUB** in format4 (though normally RSUB doesn't need extended since no address is used). Subroutine calls can now also exploit immediate addressing to call through an address constant (though typically one would use JSUB always with a label or memory operand; immediate JSUB doesn't make sense). One notable addition in SIC 2 is the possibility of a **jump to address in register** instruction via a register-to-register operation or an SVC (supervisor call) instruction, but Beck's design doesn't explicitly list a *register jump*. If needed, one could load PC via L and RSUB or use a sequence like RMO (copy register to another) into PC if it were allowed (PC isn't directly targetable by RMO in standard definitions though). Another likely addition is a **SVC** (supervisor call, sometimes opcode 0xF0 or similar) to request operating system services – if Beck included it in SIC/XE, it would trap to a monitor. However, since not explicitly in the reference, we'll omit speculation. The main improvement in control flow for SIC 2 is the ability to use base/PC relative addressing for branches (which eliminates the need for full 20-bit addresses for near jumps) and the ability to position code in the 1MB space freely with the help of base register or extended jumps for far calls.

- *Input/Output Operations:* Input/Output in SIC is handled by three dedicated instructions that perform device-level I/O one byte at a time ⁶⁷ ⁶⁸. Each device is assigned a unique 8-bit code (like an identifier for a specific I/O device or peripheral) ⁶⁸. The CPU uses these instructions in a programmed I/O manner (no DMA, no interrupts in the basic model unless one is simulating them via SW bits):

- **TD (Test Device)** – This instruction tests whether the addressed device is ready to send or receive a byte ⁶⁸. The operand of TD is the device's code (placed in the address field as an 8-bit value, often the device code is given in memory or as a constant). When executed, TD will check the status of that device's control register (simulated). If the device is ready (e.g., input available or output buffer free), the **condition code** CC in SW is set to "<" (less) to indicate *ready*; if the device is not ready (busy), CC is set to "=" (equal) ⁶⁹ ⁷⁰. (There is no "greater" for TD; presumably CC = '>' is not used for I/O, or

could indicate an error, but by convention ready = '<', not ready = '='⁶⁹.) The program can use a conditional jump after TD to wait until the device is ready (a simple busy-wait loop).

- **RD (Read Data)** – This instruction reads one byte from the device into register A⁶⁸. More specifically, it takes a device code operand (like TD) to specify which device, and if that device's data buffer has a byte ready, it transfers that byte into the rightmost 8 bits of A⁶⁸. The rest of A's bits (leftmost 16 bits) are typically unaffected or cleared – usually, the design is such that only the low byte of A is used for I/O. The CC may be implicitly set by RD or not; Beck's description suggests that after TD signals ready (CC set '<'), the program would call RD to get the data. If a device wasn't ready and one attempted RD, the behavior could be to get invalid data or simply stall until data is available (depending on the simulator's implementation – hardware might stall or have undefined behavior, so usually you do TD first).
- **WD (Write Data)** – This writes the byte contained in the rightmost 8 bits of A to the device specified⁶⁸. If the device is an output device (printer, terminal, etc.), WD will output that character or data byte. Similar to RD, one would normally check TD to ensure the device is ready to accept data (CC = '<') before executing WD. If executed when not ready, the effect might be lost data or wait.

These simple I/O instructions model the way low-level programmed I/O works on many systems (test a status bit, then read/write a data register). They are blocking in the sense that the program typically polls until done. The device codes allow multiple devices to be attached; e.g., code 00 might be the console keyboard, 01 the console printer, etc., up to 255 possible device addresses. The SIC machine doesn't define specific device addresses beyond needing them unique; it leaves that to the user's imagination or an assignment context.

SIC 2 significantly enhances the I/O capabilities by introducing the concept of **asynchronous I/O channels** that can operate in parallel with the CPU. This is a nod toward more advanced I/O architectures (like DMA or I/O channels in mainframes). In SIC 2, three additional machine instructions are defined for channel control

⁴⁰ ³⁷ :

- **SIO (Start I/O)** – This instruction starts an I/O operation on the channel or device specified. The operand (likely in A or in the instruction) identifies which device or channel to start. On execution, SIO initiates the data transfer (read or write) on that device and then *immediately returns control to the CPU without waiting for completion*. In other words, SIO launches an asynchronous I/O operation. For example, SIO F1 (if F1 were a device code in A register) might start the tape drive reading data into a buffer. SIO is analogous to a *non-blocking* I/O call. In SIC/XE, SIO is often a privileged instruction (only allowed in supervisor mode, if mode bit is implemented). After an SIO, the CPU can continue executing subsequent instructions while the device works in parallel.
- **TIO (Test I/O)** – This instruction tests the status of the I/O operation on a channel/device, similar to TD but at the channel level. It will set the condition code CC to indicate if the I/O operation has completed or if the device is still busy⁴⁰. Typically, CC = '<' might mean the I/O is finished (channel not busy), and CC = '=' means the I/O is still in progress (busy). The program can periodically execute TIO in a loop to poll for I/O completion, enabling a simple form of concurrency: the CPU does other tasks (or perhaps just loops) and checks back using TIO. This is effectively software polling; however, one could also imagine an interrupt mechanism if we use the SW's interrupt code bits.
- **HIO (Halt I/O)** – This instruction halts a currently active I/O on the specified device/channel³⁷. It might be used to abort an operation or reset a device. When executed, HIO will attempt to stop the I/O process and place the device in a known state (for example, if a disk read was in progress, HIO

might cancel it). In real hardware, not all devices can be halted easily, but as a simulation, it provides a way to terminate an I/O if needed (or it may simply set an error code to indicate termination). In the context of SIC 2, HIO could also be a way to implement a *STOP* for devices or to simulate an end-of-job.

The operands for SIO/TIO/HIO are device or channel codes. Beck's SIC/XE typically assumes a small number of *I/O channels*, where each channel could be assigned to a device. The instructions themselves might not explicitly have an operand field in the instruction stream; instead, they could implicitly use register A to specify the device/channel number (as the slides suggest: "Channel address is provided in A register" for HIO ³⁷). If so, SIO, TIO, HIO might actually be Format1 instructions (just opcodes F0, F8, F4 respectively, for example) and the convention is: *before calling SIO, load A with device ID; SIO opcode triggers that device start*. The slide references show HIO(F4) and SIO(F0) as opcodes ³⁷, which indeed implies these are standalone 1-byte instructions with an implied operand in register A. TIO isn't explicitly shown, but likely it is F8 (fits the pattern, and it was listed in scribd as well) ⁴⁰. These channel commands make the *I/O non-blocking*, which is a significant conceptual jump from SIC's purely synchronous I/O. A program can, for instance, initiate a disk read with SIO, then proceed to compute something else, and later use TIO to check if the disk read has finished, and only then use RD to actually retrieve data (or perhaps the SIO itself starts a pre-programmed transfer to memory that completes by hardware).

Execution Model and Example: Under the hood, the SIC 2 execution model for I/O assumes a separate device controller (or channel) state that operates asynchronously. In a simulation environment, one might simulate a device as a concurrent process or thread that, once started by SIO, will after some delay mark itself ready (so TIO can detect completion). This allows students to experiment with overlapped I/O and CPU activity, and introduces the need for synchronization (polling or interrupts). Some SIC/XE simulators or implementations also simulate **interrupts**: for example, an I/O device could generate an interrupt when done, which would set some code in SW and potentially transfer control to a fixed interrupt handler address. The SW's bits (interrupt code, etc. as shown by GeeksforGeeks) could facilitate that ¹⁵ ⁷¹. However, Beck's text does not delve deeply into writing interrupt-driven SIC programs – the typical usage is to poll with TIO. Still, the hardware outline is rich enough to let advanced students consider implementing a simple interrupt system if needed.

Execution Model and CPU Operation: The SIC and SIC 2 CPU follows a classic *von Neumann fetch-decode-execute cycle*. The **Program Counter (PC)** points to the next instruction to fetch ¹⁰. On each cycle, the CPU fetches the byte (or bytes) of the instruction at the PC address, decodes the opcode and flags, and fetches any additional bytes needed (since instructions can be 1, 2, 3, or 4 bytes in SIC 2). It then executes the instruction's operation, which may involve reading from memory, writing to memory, performing ALU operations, or changing the PC or registers. After execution, the PC is normally advanced to point to the next instruction (PC += instruction_length, which is 3 for format3, 4 for format4, etc., except in case of jumps where PC is loaded with a new value) ⁷² ⁷³. The machine does not implement pipelining; each instruction is processed to completion before the next begins. Given the simplicity, one can think of it as a single-cycle-per-byte or a multi-cycle (but sequential) architecture. In educational simulators, this is often visualized as stepping through instructions one by one.

During execution of memory-reference instructions, the effective address (EA) is calculated considering the flags: e.g., if immediate, EA is just (the value from address field); if indirect, the CPU will do an extra memory fetch. If indexed, adds X. If base relative, adds B, etc. Once the EA or operand value is determined, the CPU performs the operation (like add, compare, move, etc.). For store instructions, it writes to memory at EA; for load, it reads from memory at EA. For immediate mode, it might not access memory at that EA at all but use

the EA itself as operand. All of this logic constitutes the *micro-operation* sequence of each instruction. The architecture's description is typically at the macro level (the effect of each instruction), leaving the micro-level sequencing conceptually to the student's understanding or to a microprogram if one is implementing it in an FPGA. A block diagram of a SIC 2 CPU would show an ALU, a register file (A, X, L, ...), a mechanism to add X or B to addresses, and logic to handle the n,i,b,p,e bits (e.g., a control unit that decides how to form the effective address). Indeed, one academic project implemented a SIC/XE CPU on an FPGA, complete with a memory controller and device controllers ⁷⁴ ⁷³. In that design, the CPU had a 20-bit address bus to memory (for the 1MB space) and 8-bit data buses for memory I/O, since memory was byte-oriented ⁷² ⁷³. The datapath included an ALU and various multiplexers to choose between PC or computed addresses, etc. ⁷³. The control unit would handle states like instruction fetch, decode, operand fetch, execute, result store, similar to any CPU.

One notable aspect of the execution model is **subroutine linkage**. Since SIC has no stack registers, subroutine calls via JSUB/RSUB mean that a program cannot easily do nested subroutine calls without saving the L register. A typical SIC calling convention might be: before calling a second subroutine from within a subroutine, save L (e.g., by STL to memory), then do JSUB. On return, restore L from memory. This is burdensome compared to having a hardware stack, but it illustrates how link registers or "jump-to-subroutine with link" works (similar to the BLR in early IBM or the JSR in some RISC with link register). SIC 2 does not change this mechanism – it still relies on L and explicit saves for nesting. This limitation itself can be a teaching point (it emphasizes the role of a stack pointer or how recursion would require software handling).

Example Program Scenario: To illustrate SIC 2 instruction usage, consider a simple task: compute the sum of an integer array and print the result. In SIC 2 assembly, one might do:

- Initialize sum to 0 in register A (e.g., `CLEAR A` if available, or `LDA =0` using an immediate constant).
- Use register X as an index through the array, clear X to 0. Use register S to hold the array's length (loaded from a known memory location or immediate).
- Loop: use `LDT ARRAY, X` (if T is used as a temporary) or simply `LDB ARRAY` to set B as base and then use `LDA 0, B` via base-relative – but more straightforward: `LDA ARRAY, X` (direct indexed) to load the next element. Then `ADDR A, T` (if T held sum; but we used A for sum, so instead do `ADD ARRAY, X` directly adding from memory to A). Actually, in SIC (and XE) `ADD` can directly add from memory to A, so simply doing `ADD ARRAY, X` will add the word at address (ARRAY + X) to A ⁷⁵.
- After adding, increment X by the word size. In SIC, you might do `LDT #3` (to load literal 3) and then `ADDR X, T` to add 3 to X. Or use `TIX` if we set a limit. Alternatively, use `TIX ARRAYEND` approach: If we know the number of iterations, we can set the operand of TIX to the array length * 3 (end address offset). But a simpler approach: Use X as a counter of elements rather than byte offset, since TIX compares X to a memory value. For instance, load X with 0, memory location N with length (count), then do `TIX N` each time. TIX increments X by 1 and sets CC based on X vs N ⁶⁴. If CC != < (meaning X reached N), break the loop. But we need to add the correct array element: with X incrementing by 1, we could multiply X by 3 to get a byte offset – that's not directly done, so perhaps better to keep X as byte offset and compare differently. Actually, one can use `TIXR` in XE: if we load S with N*3 (the byte length), we can do `TIXR S` to increment X by 1 and compare X to S, but X here should count bytes/words? The assembly might require careful thinking. In a classic SIC solution, one

might increment X by 3 manually each loop and use a counter. Let's not digress; the key is, these instructions allow iteration with either TIX or simple arithmetic on X.

- After loop, the sum is in A. To print it, in SIC you would output byte by byte via dividing by 10 to get digits, etc., or simply store it somewhere for an output routine. With SIC 2 channel I/O, one could imagine writing the sum to a buffer and using SIO to print, but that's elaborate. Typically, you'd do: convert number to ASCII in memory, then use WD in a loop to output characters, each time checking TD for the output device. If an OS were present, one might do SVC to request a print service.

This scenario shows how SIC 2 instructions map to typical tasks. The architecture's simplicity means things like multiplication of index by word size are manual (no scaled indexing), and I/O is low-level.

Execution Model and CPU Features

The execution model of SIC 2, as touched on, is sequential and fairly simple by design. Each instruction is executed to completion before the next begins (there is no pipelining, out-of-order execution, or parallel instruction issue). This simplicity makes it easier to reason about in software simulations and educational settings – the outcome of each step is deterministic and can be stepped through easily in a debugger.

One can describe the CPU operation in terms of *phases*: **Fetch** the instruction at PC, **Decode** it (determine format and opcode, identify if there is a target address to compute), **Fetch Operands** (if it's a memory or register operand, gather the data: possibly reading memory, or reading a register, or resolving an immediate), **Execute** (perform the ALU operation or control action), and **Write-back** (store the result to a register or memory if needed, and update PC for the next instruction). This is essentially a finite-state machine cycle that can be microprogrammed.

The **Status Word (SW)** deserves a bit more elaboration in execution. We know CC is part of SW, and it's affected by COMP, TIX, TD/TIO results. In some implementations, SW may also hold a *mask* of allowed interrupts and a bit indicating the CPU's mode (user or supervisor) ⁷⁶. For example, a student project or an instructor might augment the SIC/XE specification such that bit 15 of SW is the interrupt mask, and bits 8–11 encode an interrupt vector when an interrupt occurs ¹⁵. If an I/O device finishes and interrupts, it could place an code in SW (ICode bits [16–23] per GeeksforGeeks) ⁷⁷ and switch CPU to supervisor mode (mode bit set to 1) to handle it. This goes beyond Beck's initial scope but is a natural extension if one wants to simulate an OS.

Interrupt Handling (Optional in SIC 2): If implemented, an interrupt would likely work as follows: When a device completes and if interrupts are enabled, the CPU would automatically trap to a fixed location (say address 0 or some predefined interrupt handler address), similar to how some simple architectures do. The current PC and maybe SW could be saved (perhaps in reserved memory or at a memory stack if one is simulated). The mode bit in SW would switch to supervisor mode, and the interrupt code field would indicate which device or cause triggered it ¹⁵ ⁷⁷. The OS handler would then service the interrupt (e.g., read the data or mark I/O complete), and issue an RFI (return from interrupt) which could be perhaps implemented via an **LPS** (Load Processor Status) to restore SW including the mode and CC and a jump back. But since Beck's main focus was system software, not hardware interrupts, this is typically an advanced topic possibly left as an exercise. More commonly, SIC programs explicitly check device status with TD/TIO.

Memory Protection and Privilege: The SIC 2 architecture, by default, does not enforce memory protection or privileged instructions aside from perhaps making SIO/TIO/HIO privileged (the presence of a supervisor

mode bit suggests such a convention: mode=0 user, mode=1 supervisor, and maybe channel I/O can only run in supervisor mode) ⁷⁶. In an educational OS, one could implement a simple separation: user programs run with mode=0 (so if they try a privileged op like SIO, the hardware traps them), and an OS runs with mode=1 and can execute those. However, this is all conceptual – the actual SIC/XE as presented in the textbook doesn't formalize it. It's an opportunity for instructors to simulate it if desired.

Performance considerations: Since there is no pipelining, each instruction's length influences its execution time (in a simulation, you might count cycles equal to number of memory bytes accessed, etc.). Format4 instructions take an extra fetch cycle due to the fourth byte. Indirect addressing takes an extra memory access (to fetch the pointer). These details can be pointed out to simulate aspects of real CPU performance. But at the level of usage, one doesn't usually measure performance of SIC programs except perhaps to illustrate how loop unrolling or using registers can reduce instruction count.

Enhancements of SIC 2 over SIC (Summary of Improvements)

SIC 2 (SIC/XE) introduced a number of enhancements to overcome the limitations of the original SIC architecture. Here we summarize the key improvements and their significance ⁵⁵ ⁷⁸ :

- **Expanded Memory:** The original SIC is limited to 32K bytes of memory, whereas SIC 2 expands the addressable memory to 1 MB ²¹. This is a jump from 15-bit to 20-bit addressing. The larger memory enables more complex programs and larger data sets to be loaded, and it necessitated new instruction formats (formats 3 and 4) to handle 20-bit addresses via base relative and extended addressing. This makes SIC 2 capable of handling more realistic workloads and scenarios (e.g., multiprogramming with each program in a different region of memory).
- **Additional Registers:** SIC 2 adds four new registers (B, S, T, F) to the original five ¹⁶. The extra general-purpose registers (S, T) allow holding more intermediate values or pointers, reducing the need to use memory for temporary storage. The Base register (B) unlocks new addressing modes (base-relative) which improve code modularity and relocatability. The Floating-point accumulator (F) introduces a new computation domain (real numbers). More registers also facilitate compiler register allocation in hypothetical high-level language implementations targeting SIC 2, thus bridging a gap toward modern register-rich architectures.
- **Floating-Point Arithmetic:** Absent in SIC, hardware floating-point operations become available in SIC 2 via the 48-bit F register and corresponding opcodes (ADDF, SUBF, MULF, DIVF, etc.) ²⁶ ²⁷. This enhancement allows students to experiment with non-integer arithmetic and understand the complexity of floating-point representation (range and precision, exponent bias, etc.). It also demonstrates how an architecture can be extended with a new data type in a backward-compatible way (integer programs still run unchanged, but new code can utilize floats).
- **Instruction Formats and Addressing Modes:** SIC 2's introduction of multiple instruction formats (including shorter 1-byte and 2-byte instructions) and the suite of addressing mode bits (n,i,x,b,p,e) is arguably the most significant enhancement. It brings in *immediate and indirect addressing*, *PC-relative and base-relative addressing*, and *extended format* support ⁷⁹ ⁴⁸. These collectively make the instruction set far more expressive. Programs can be more efficient – immediate mode avoids extra memory loads for constants, PC-relative mode makes branching and data access position-independent and compact, and base-relative mode allows program relocation and larger program sizes without paying the cost of 4-byte addresses everywhere ⁵⁵. The extended format ensures that even with these compact modes, the architecture can still reach any address in the larger memory when necessary. All these modes were chosen to be *upward-compatible* with SIC's original scheme:

any valid SIC addressing usage is representable within the new flags (often with $n=1, i=1$ and appropriate b/p flags) ⁴¹. This design is a lesson in how to extend an ISA without breaking existing code.

- **Extended Instruction Set:** In addition to new addressing modes, SIC 2 defines many new opcodes, expanding the operation repertoire. We have already mentioned the floating-point and register-to-register arithmetic instructions. Other additions include **LPS** (load processor status word), **STI** (store instruction? possibly not, maybe this is a guess), and certainly the **register move** (RMO) and **clear** and **compare registers** instructions. The machine control instructions – **SIO**, **TIO**, **HIO** – are new in SIC 2 and greatly enhance I/O management ⁵⁵ ⁴⁰. There might also be **SHIFT** instructions (some implementations of SIC/XE include SHIFTL/SHIFTR, but Beck's core description did not emphasize them; however, given format2's free space, some assemblers define SHIFT opcodes). If included, shift/rotate instructions would allow bit manipulation for algorithms like encryption, checksums, etc., more easily than relying on repeated additions. In summary, SIC 2's instruction set is a superset of SIC's: everything SIC could do, SIC 2 does as well or better, and it adds dozens of new capabilities making it comparable to a simple modern CPU.
- **Parallel I/O and Device Support:** By allowing asynchronous I/O operations (through SIO/TIO/HIO and possible interrupt handling), SIC 2 moves beyond the purely sequential, CPU-bound I/O of SIC ⁵⁵. This models how real computers deal with slow I/O devices by overlapping computation and I/O. In educational terms, this allows demonstration of device independence (you can start an I/O and meanwhile compute something else) and teaches the need for synchronization (polling or interrupt to know when I/O completes). It also implicitly introduces the concept of *privileged operations* – the idea that not all programs should directly manipulate devices – if the mode bit in SW is used to restrict these instructions to an OS.
- **Upward Compatibility and Migration Path:** An often overlooked but crucial enhancement is that SIC 2 was designed to run SIC programs unchanged ³ ⁸⁰. All original SIC instructions are still valid in SIC 2, and their opcodes produce the same effects. The assembler for SIC 2 can accept SIC source programs, and the machine will interpret them correctly (since $n=0, i=0$ for legacy opcodes is handled as direct addressing mode). This meant instructors and students could start with SIC in early chapters (covering basic assembler design, etc.), and then progress to SIC 2 (XE) in later chapters for advanced topics, without discarding prior work. It also reflects how real ISAs evolve (new generations maintain backward compatibility). Upward compatibility ensured that any educational tools (assemblers, simulators) built for SIC could be extended to support SIC 2 gradually.

To encapsulate the improvements: SIC 2 transforms the simplistic SIC into a far more practical architecture suitable for demonstrating real-world system software concepts. It bridges the gap between an overly simple toy and a full-scale production architecture. Features like indexing, base registers, and PC-relative addressing connect directly to concepts in assemblers (like addressing modes and program relocation) ⁵² ⁵³. Floating-point and additional registers connect to compiler/code generation topics (how to use registers efficiently, how floating point is handled). Asynchronous I/O and status flags connect to operating system design (interrupts, concurrency, device drivers). In sum, SIC 2 is a deliberately pedagogical enhancement to the original SIC, incorporating lessons from real CPU evolutions while keeping the system straightforward enough to grasp in a classroom setting.

Relevance in Education, Simulation, and Emulator Projects

The SIC and SIC 2 architectures have proven to be **invaluable pedagogical tools** in computer science and engineering education. Their design strikes an effective balance between simplicity and functionality, making them ideal for illustrating core concepts of computer architecture and system software:

- **Computer Architecture & Organization Education:** SIC provides a simplified model of a computer, with clearly defined CPU, memory, and I/O components. By studying SIC (and SIC 2) architecture, students learn about the basic hardware structure—how memory is addressed, how registers function, and how a CPU executes instructions ⁸¹. The SIC 2 extension introduces students to more sophisticated architectural features (like additional registers and addressing modes) without the overwhelming complexity of a modern x86 or ARM processor. Educators often use SIC as an introductory example to explain how instructions are encoded and executed, how an addressing mode works, or how a simple CPU datapath might be structured. Because the architecture is completely documented and hypothetical, students can be tasked with thought exercises like modifying the instruction set or extending the architecture themselves, which encourages deep understanding.
- **Assembly Language Programming Practice:** The SIC assembly language is straightforward and small, making it an excellent first assembly language for students to learn. It includes just enough instructions to perform typical programming tasks (loops, arithmetic, subroutine calls, I/O) so that students can write meaningful programs (sorting a list, computing a formula, reading input/output) without getting lost in syntax or peculiarities of a real assembly. The simplicity of SIC's instructions (especially in the standard version) helps beginners grasp how high-level constructs map to machine operations. As they progress, SIC 2's assembly exposes them to immediate values, base-relative addressing for data structures, and other patterns common in assembly for real architectures. Many courses have students implement projects like an *assembler for SIC/SIC 2* or write small assembly routines to solidify their understanding ⁸² ⁸³. The upward compatibility means student-written SIC programs can later be run on a SIC 2 simulator to observe differences in performance or to integrate with new instructions. Overall, SIC's assembly is praised for being “simple and easy to understand, making it a useful tool for teaching assembly language programming” ⁸⁴. Students can focus on concepts like addressing modes and instruction effects rather than arcane assembler directives or huge opcode sets.
- **System Software (Assemblers, Linkers) Implementation:** Perhaps the most significant use of SIC/SIC 2 is in system software courses, where students build components like assemblers, linkers, and loaders. Beck's textbook itself guides the reader through constructing a two-pass assembler for SIC/XE as a case study. Because SIC has a well-defined instruction format and a manageable number of opcodes, writing an assembler for it is feasible as a semester project. Students learn how to translate assembly code (with labels, directives like BYTE/WORD, maybe even simple literals) into machine code and how to manage symbol tables, addresses, and relocations. The presence of different addressing modes and program relocation (using base relative and PC relative) directly ties into how a relocating loader or linker must work – for example, resolving addresses that can't be determined until load time and generating relocation records for format4 addresses or immediate values beyond range. Many open-source and academic projects have been done to implement SIC/XE assemblers in various languages ⁸² ⁸⁵, reflecting how common this educational exercise is. Similarly, writing a loader that can load SIC object code (with modification records for relocated addresses) is another

assignment – simplified by SIC’s straightforward relocation scheme. The outcome is that SIC/XE gives students a taste of implementing real system software components that are directly analogous to those for real machines, but targeting a machine that they can fully understand.

- **Compiler and OS Prototyping:** Although less common than assemblers, SIC 2 has been used in teaching compilers and operating systems at a conceptual level. For compilers, one can use SIC 2 as a target architecture for code generation. Since SIC 2 has features like registers and indexed addressing, students can design simple code generation algorithms from a high-level language to SIC 2 assembly ⁸⁶. It’s enough to illustrate concepts like expression translation, register allocation (especially given limited registers, which is a realistic scenario), and calling conventions (handling subroutine calls with L register, etc.). The small scale keeps the focus on the core ideas of compilation rather than the quirks of a real ISA. For operating systems, SIC 2 can act as a sandbox where students implement rudimentary OS functionalities such as scheduling, memory management, or I/O drivers. For instance, because SIC 2 can simulate I/O interrupts and has a mode bit (in some implementations), it’s possible to demonstrate *process switching*: a user program doing I/O might trigger an interrupt, the OS (running in supervisor mode) takes over, saves context (registers, including PC and SW), and switches to another task – all in a controlled environment. Students can implement simple round-robin scheduling or memory allocation on a SIC/XE simulator, thus learning OS concepts without needing an actual hardware platform. GeeksforGeeks notes that “the SIC’s simple architecture can be used as a basis for teaching operating system development” and that students can learn about process management, memory management, and I/O management by implementing them on SIC ⁸⁷. This is a powerful statement – it means that despite being a hypothetical machine, SIC is versatile enough to host a miniature OS, reinforcing concepts with hands-on experimentation.
- **Simulation and Emulation Projects:** The well-defined simplicity of SIC/SIC 2 also makes it a popular target for simulation and emulation projects. Writing a **SIC virtual machine (VM)** or interpreter is a common exercise that solidifies understanding of instruction execution. There are numerous student-created and open-source SIC simulators (in C, C++, Java, Python, etc.) that emulate the SIC/XE instruction set ⁸² ⁸⁸. Such a project teaches how to implement fetch-decode-execute, handle memory and I/O, and possibly how to debug an ISA implementation. It’s manageable in scope – one can implement all instructions and a simple console I/O device in a few hundred lines – yet it’s complex enough to be interesting (especially with SIC 2 features like multiple formats and addressing modes which make the decoding logic non-trivial). These simulators allow users to run SIC assembly programs and observe the CPU state step-by-step, which is excellent for learning. Additionally, some ambitious projects have involved implementing SIC/XE on hardware description languages or FPGA boards ⁶. For example, a diploma thesis in 2011 by Kloboves implemented a SIC/XE processor on a Digilent FPGA, including device controllers and a connecting interface to a PC ⁶ ⁷⁴. This demonstrates that SIC 2 is simple enough to be realized on an FPGA with limited resources, and doing so teaches digital design, FSM control, and microarchitecture. The resulting “SIC chip” on FPGA can execute SIC machine code natively at some clock speed, effectively turning the hypothetical into a tangible, if simplified, CPU. The educational payoff is seeing how higher-level language code (or assembly) translates down to logic gates and flip-flops that implement the architecture. One can experiment with microprogramming or hardwired control using the SIC design, and try modifications like pipeline it or adding cache as further exercises.

- **Community and Resources:** Over the years, a community of instructors and students has built around SIC/SIC 2. There are presentations, slides, and online tutorials explaining the architecture, as well as solved examples and even debugging tools. The consistency of Beck's textbook through multiple editions means many people share similar assignments and solutions, creating a wealth of reference material. For instance, the GeeksforGeeks articles ² ⁸⁹, slide decks on Slideshare ⁹⁰, and various GitHub repositories all contribute to accessible learning resources for new generations of students. This collaborative knowledge base underscores SIC 2's relevance: it remains a go-to example in system software curricula internationally ⁵.

Conclusion: The SIC 2 chip (architecture) may be simple and hypothetical, but it encapsulates fundamental principles of computer architecture in a form that is exceptionally conducive to teaching and exploration. Its design includes just enough complexity to be realistic – multiple addressing modes, a variety of instructions, I/O concurrency – while being minimal enough that students can master it and even implement it themselves. In advanced courses, SIC 2 serves as a stepping stone: once students understand how SIC 2 works, they can more easily appreciate the design decisions and challenges in real architectures (like x86's complexity or MIPS's RISC approach), because they have built a mental model from the ground up. In simulation environments, SIC 2 provides a playground for experimenting with system-level code safely. And in hardware projects, it offers a blueprint of a small CPU that can be brought to life, demystifying how machines execute the software we write.

In sum, SIC 2 has endured as a relevant educational architecture because it elegantly bridges theory and practice. It was born out of a need to simplify and teach, and over time it has evolved (through SIC/XE and various community extensions) to remain a comprehensive teaching tool. Whether in software simulation or as an FPGA “chip,” SIC 2 continues to illuminate the workings of computer systems for students, providing a foundation of knowledge that they carry into understanding real-world computing systems.

Sources: The information in this paper is drawn from Leland Beck's description of the SIC/XE architecture and various educational resources that detail the machine's specifications and uses. Key references include Beck's *System Software* textbook and summaries of SIC architecture ⁹¹ ⁹² ⁶⁷ ⁵⁵, which provide the definitive specs for registers, memory, and instruction formats, as well as academic projects and articles that reflect on the application of SIC/SIC 2 in teaching and simulation ⁵ ⁸⁹. These sources collectively reinforce the technical details and the pedagogical value of the SIC 2 architecture as presented above.

¹ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²¹ ⁹¹ ⁹² Simplified Instructional Computer - Wikipedia

https://en.wikipedia.org/wiki/Simplified_Instructional_Computer

² ³ ⁴ ⁸ ⁹ ¹⁰ ¹⁴ ¹⁵ ⁷¹ ⁷⁶ ⁷⁷ ⁸⁰ ⁸¹ ⁸⁴ ⁸⁶ ⁸⁷ ⁸⁹ Simplified Instructional Computer (SIC) - GeeksforGeeks

<https://www.geeksforgeeks.org/simplified-instructional-computer-sic/>

⁵ ⁶ ⁷² ⁷³ ⁷⁴ Implementacija procesorja SIC/XE na FPGA in podporna sistemska programska oprema

https://ucilnica.fri.uni-lj.si/pluginfile.php/11881/mod_label/intro/KlobovesK.pdf

⁷ ²⁰ ²⁴ ²⁶ ²⁷ ²⁸ ²⁹ ³¹ ³² ³⁹ ⁴⁰ ⁴⁹ ⁵⁵ ⁶⁷ ⁶⁸ ⁷⁵ ⁷⁸ ⁷⁹ SIC Architecture | PDF

<https://www.scribd.com/presentation/60202263/SIC-Architecture>

11 12 13 22 30 33 34 35 36 37 38 41 42 43 44 45 46 47 48 50 51 52 53 54 56 57 58 59 60 61

62 63 64 65 66 69 70 90 **system software.ppt**

<https://www.slideshare.net/slideshow/system-softwareppt/256888126>

23 25 **Chapter 1-Background**

http://enggnotebook.weebly.com/uploads/2/2/7/1/22718186/chapter_1backgroundnew.pdf

82 **chimms1/SIC-Assembler - GitHub**

<https://github.com/chimms1/SIC-Assembler>

83 **shaimawaz/SICassembler: This is a project that assembles a SIC file ...**

<https://github.com/shaimawaz/SICassembler>

85 **viplavkumar96/SIC-Assembler - GitHub**

<https://github.com/viplavkumar96/SIC-Assembler>

88 **rohitthapliyal2000/SIC-Assembler: C++ implementation of ... - GitHub**

<https://github.com/rohitthapliyal2000/SIC-Assembler>